

# MACHINE LEARNING — LAB MANUAL

## Lab 7: Naive Bayes — Spam Detection & Sentiment Analysis

Week 7 — Fall 2026 — BS Computer Science, 5th Semester

**Duration: 3 Hours in-session.**

Prerequisites: Labs 1–6 (Python/tools, cleaning/EDA, Decision Trees, Random Forest, Logistic Regression, KNN); Naive Bayes theory (Bayes' theorem, conditional independence assumption, Multinomial/Gaussian/Bernoulli variants) from the ML course

Tools Required: Python 3.x, Pandas, NumPy, scikit-learn, Matplotlib, Google Colab, a Kaggle account

Includes: Text cleaning & vectorization, two full Naive Bayes classification tasks, word-probability interpretation, two Kaggle Notebook publications, a cross-task comparison, and a graded rubric

### Learning Objectives

This lab is NOT a re-introduction to Naive Bayes or Bayes' theorem — you already know the math and the independence assumption from your ML course. The goal is to practice the full text-classification workflow end to end, on two different real problems, and to publish your work on Kaggle for the first time in this course. By the end of this lab you should be able to independently:

- Clean and vectorize raw text into numeric features (Bag-of-Words / TF-IDF)
- Train and evaluate a Multinomial Naive Bayes classifier for text classification
- Reason about which evaluation metric matters most for a specific real-world cost trade-off (e.g. a false positive vs. a false negative in spam filtering)
- Interpret a Naive Bayes model by inspecting its learned word probabilities
- Transfer a working pipeline to a second, different text classification problem with minimal guidance
- Publish a working notebook publicly on Kaggle, attached to a real dataset
- Compare Naive Bayes against Logistic Regression (Lab 6) on the same text features

| Module | Focus Area                                                |
|--------|-----------------------------------------------------------|
| 1      | Problem Definition & Dataset Exploration (0–15 min)       |
| 2      | Task A — Text Cleaning & Vectorization: Spam (15–40 min)  |
| 3      | Task A — Naive Bayes Model & Evaluation: Spam (40–65 min) |
| 4      | Task A — Word Interpretation: Spam (65–80 min)            |
| 5      | Task A — Kaggle Notebook Submission (80–90 min)           |

| Module | Focus Area                                                       |
|--------|------------------------------------------------------------------|
| 6      | Task B — Text Cleaning & Vectorization: Sentiment (90–115 min)   |
| 7      | Task B — Naive Bayes Model & Evaluation: Sentiment (115–140 min) |
| 8      | Task B — Kaggle Notebook Submission (140–150 min)                |
| 9      | Cross-Task Comparison & vs. Logistic Regression (150–165 min)    |
| 10     | Final Reflection & Conclusion (165–180 min)                      |

## Datasets & Problem

This lab moves from the tabular data you've used since Lab 2 to text data — a genuinely different kind of feature space, and exactly where Naive Bayes traditionally shines. You will complete two independent classification tasks on two real Kaggle datasets, using the same underlying pipeline (clean → vectorize → train Naive Bayes → evaluate → interpret) both times. Task A walks you through every step in detail; Task B asks you to apply the same pipeline with much less guidance, to test whether the skill actually transfers.

| Item                 | Task A — Spam Detection                                                                                                           | Task B — Sentiment Analysis                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset name         | SMS Spam Collection                                                                                                               | Twitter US Airline Sentiment                                                                                                              |
| Official Kaggle link | <a href="https://kaggle.com/datasets/uciml/sms-spam-collection-dataset">kaggle.com/datasets/uciml/sms-spam-collection-dataset</a> | <a href="https://kaggle.com/datasets/crowdflower/twitter-airline-sentiment">kaggle.com/datasets/crowdflower/twitter-airline-sentiment</a> |
| Size                 | 5,572 SMS messages                                                                                                                | 14,640 tweets (use positive/negative only — drop neutral for this lab)                                                                    |
| Target               | label: spam / ham                                                                                                                 | airline_sentiment: positive / negative                                                                                                    |

### Before You Start

- Create a free Kaggle account if you don't already have one — you will need it for Sections 5 and 8.
- Download both datasets from the links above (spam.csv and Tweets.csv) and upload them to your Colab session.
- Create a new Colab notebook: week7\_<yourname>\_naive\_bayes.ipynb, with the student-info block and Markdown headings described in Submission Requirements.
- Work through the sections in order. If you run out of time in the 3-hour session, Task A (Sections 1–5) is the priority to finish in class — complete Task B and the remaining sections at home before your final submission.

## Part 1 — Problem Definition & Dataset Exploration

Before cleaning or modeling anything, understand both problems side by side: what a single row represents, what the target is, and how balanced each dataset's classes are — this sets expectations for evaluation later in both tasks.

### Concepts / Functions You May Find Useful

| Concept / Function                                             | Purpose                                      |
|----------------------------------------------------------------|----------------------------------------------|
| <code>pd.read_csv(...)</code>                                  | Load spam.csv and Tweets.csv                 |
| <code>df.shape / df.head() / df['label'].value_counts()</code> | Inspect size, sample rows, and class balance |

## Tasks

1. Load both datasets. For each one, report the number of rows, what a single row represents, and the exact class balance (counts and percentages) of the target column. Keep only the positive and negative tweets for the sentiment dataset going forward — drop neutral rows now, and state how many rows that leaves you with.
2. In 2–3 sentences per dataset, state the practical problem being solved (what decision would this model actually support?) and why each one is a binary classification problem.

## Part 2 — Task A: Text Cleaning & Vectorization (Spam)

Naive Bayes (and every other classifier) needs numbers, not raw sentences. Bag-of-Words / TF-IDF convert each message into a row of word-frequency counts, where every unique word in the vocabulary becomes its own feature/column — a completely different kind of feature space than Labs 2–6's structured columns.

### Concepts / Functions You May Find Useful

| Concept / Function                                                                        | Purpose                                                               |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>text.str.lower()</code>                                                             | Normalize case so 'Free' and 'free' aren't treated as different words |
| <code>from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer</code> | Import text-vectorization tools                                       |
| <code>vectorizer.fit_transform(X_train) / vectorizer.transform(X_test)</code>             | Fit the vocabulary on training data only, then apply to both sets     |
| <code>vectorizer.get_feature_names_out()</code>                                           | See the actual vocabulary words the vectorizer learned                |

## Tasks

3. Lightly clean the message text (lowercase at minimum; consider removing punctuation). Keep this simple — heavy NLP preprocessing is not the point of this lab.
4. Encode the label column as 0/1, split into training and test sets, then fit a `CountVectorizer` or `TfidfVectorizer` on the training messages only and transform both sets. Report the resulting vocabulary size (number of features) and explain in 1–2 sentences why the vectorizer must be fit only on training data, not the full dataset.

## Part 3 — Task A: Naive Bayes Model & Evaluation (Spam)

Train a Multinomial Naive Bayes classifier — the standard choice for word-count/TF-IDF features — and evaluate it exactly as you evaluated every other classifier since Lab 3, plus one new, genuinely important question specific to spam filtering: which kind of mistake is worse?

### Concepts / Functions You May Find Useful

| Concept / Function                                         | Purpose               |
|------------------------------------------------------------|-----------------------|
| <code>from sklearn.naive_bayes import MultinomialNB</code> | Import the classifier |
| <code>MultinomialNB().fit(X_train_vec, y_train)</code>     | Trains the model      |

| Concept / Function                                                        | Purpose                                  |
|---------------------------------------------------------------------------|------------------------------------------|
| accuracy_score, precision_score, recall_score, f1_score, confusion_matrix | Same evaluation tools as every prior lab |

## Tasks

- Train a MultinomialNB model on the vectorized training data. Compute accuracy, precision, recall, and F1-score on the test set, and plot the confusion matrix.
- A false positive here means a real (ham) message gets wrongly flagged as spam and possibly hidden from the user; a false negative means a spam message gets through to their inbox. In 3–4 sentences, argue which kind of error is worse for this specific application, and therefore which metric (precision or recall) you would prioritize if you had to tune the model further.

## Part 4 — Task A: Word Interpretation (Spam)

Naive Bayes is interpretable in its own way: it learns a probability for every word given each class. Inspecting these probabilities is this lab's equivalent of Lab 5/6's coefficient tables — a direct look at what the model actually learned.

### Concepts / Functions You May Find Useful

| Concept / Function      | Purpose                                                                |
|-------------------------|------------------------------------------------------------------------|
| model.feature_log_prob_ | $\log P(\text{word} \mid \text{class})$ for every word, for each class |
| np.argsort(...)         | Find the indices of the highest/lowest values in an array              |

## Tasks

- Using `feature_log_prob_`, find and list the 15 words most strongly associated with spam (highest log-probability difference between the spam and ham classes) and the 15 most associated with ham. Do these words match your intuition about what spam looks like?
- Write 2–3 of your own test messages (one obviously spam-like, one obviously not) and run them through your trained pipeline. Do the predictions match what you expected? If not, discuss why in 1–2 sentences.

## Part 5 — Task A: Kaggle Notebook Submission

Neither dataset in this lab is an active Kaggle competition with a leaderboard, so "submitting" here means publishing your working notebook publicly on Kaggle, attached to the dataset — a real, common way ML practitioners share their work and build a public portfolio. This is the same platform skill as a leaderboard submission, just without a rank.

## Tasks

- On the SMS Spam Collection dataset's Kaggle page, click "New Notebook" — this creates a Kaggle-hosted notebook already attached to the dataset. Recreate (or paste) your Task A cleaning, vectorization, model,

evaluation, and word-interpretation cells there, and run every cell top to bottom to confirm it works on Kaggle's environment too, not just in Colab.

10. Add a short Markdown summary at the top of the Kaggle notebook (2–3 sentences: what the notebook does and your headline result), then use "Save Version" → "Save & Run All (Public)" to publish it. Record the public notebook URL in your Colab notebook's Conclusion section.

## Part 6 — Task B: Text Cleaning & Vectorization (Sentiment)

Repeat Section 2's pipeline on the airline tweets — but this time with far less guidance. You already know the steps; this section tests whether you can apply them to a new, messier dataset (tweets contain @mentions, links, and hashtags that SMS messages didn't) on your own.

### Concepts / Functions You May Find Useful

| Concept / Function                                     | Purpose                                                    |
|--------------------------------------------------------|------------------------------------------------------------|
| <code>text.str.replace(pattern, "", regex=True)</code> | Strip out @mentions / links if you choose to               |
| Same vectorizer classes as Section 2                   | CountVectorizer or TfidfVectorizer — your choice this time |

### Tasks

11. Using the positive/negative tweets you kept from Part 1, clean the text (decide for yourself how much cleaning is worth doing — at minimum handle lowercasing; consider whether @mentions or links are useful signal or noise) and vectorize it, following the same fit-on-train-only discipline as Task A.
12. Briefly justify (1–2 sentences) any cleaning decisions that go beyond what you did for the SMS messages, and why tweets might need them.

## Part 7 — Task B: Naive Bayes Model & Evaluation (Sentiment)

Train and evaluate Naive Bayes on the sentiment data exactly as in Part 3, then step back and compare how this task went relative to spam detection.

### Concepts / Functions You May Find Useful

| Concept / Function                                                                                                                             | Purpose                         |
|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>MultinomialNB()</code>                                                                                                                   | Same classifier as Task A       |
| <code>accuracy_score</code> , <code>precision_score</code> , <code>recall_score</code> , <code>f1_score</code> , <code>confusion_matrix</code> | Same evaluation tools as Task A |

### Tasks

13. Train a MultinomialNB model on the vectorized tweet data. Compute accuracy, precision, recall, and F1-score, and plot the confusion matrix.
14. Compare this task's metrics to Task A's. Did Naive Bayes do better or worse on sentiment than on spam? In 3–4 sentences, connect this to the "naive" independence assumption specifically — is treating every word as

independent more or less reasonable for short, formulaic spam text versus tweets, where sarcasm, negation ("not good"), and context matter more?

## Part 8 — Task B: Kaggle Notebook Submission

---

Repeat Part 5's publishing process for the sentiment task, on the Twitter US Airline Sentiment dataset's own Kaggle page.

### Tasks

15. Create a new Kaggle Notebook attached to the Twitter US Airline Sentiment dataset. Recreate your Task B cleaning, vectorization, model, and evaluation cells there, run them top to bottom, add a short Markdown summary, and publish it the same way as Part 5.
16. Record this second public notebook URL in your Colab notebook's Conclusion section alongside the first.

## Part 9 — Cross-Task Comparison & vs. Logistic Regression

---

You now have two Naive Bayes results and, from Lab 6, experience with Logistic Regression on scaled tabular features. Here you extend the comparison habit from Labs 4 and 6 to text data: does Naive Bayes's advantage (if any) hold up against a different algorithm on the same features?

### Concepts / Functions You May Find Useful

| Concept / Function                             | Purpose                                                                  |
|------------------------------------------------|--------------------------------------------------------------------------|
| <code>LogisticRegression(max_iter=1000)</code> | Train on the same TF-IDF/Bag-of-Words features as your Naive Bayes model |
| <code>pd.DataFrame({...})</code>               | Assemble comparison tables                                               |

### Tasks

17. Build a table comparing Naive Bayes on Task A vs. Task B (accuracy, precision, recall, F1) side by side.
18. Train a Logistic Regression model on Task A's same vectorized features and compare it against your Task A Naive Bayes results in a second table. Which model wins, and does that match what you'd expect given what Naive Bayes assumes about word independence?

## Part 10 — Final Reflection & Conclusion

---

Step back from both tasks and summarize what you've learned about Naive Bayes, text classification, and how it fits alongside the four tabular classifiers from Labs 3–6.

### Tasks

19. Write a short conclusion (6–8 sentences) covering: which task suited Naive Bayes better and why; what surprised you about the most predictive words in Part 4; how Naive Bayes compared to Logistic Regression on

the same features; and one limitation of the "naive" independence assumption you personally observed in your results.

20. List both of your published Kaggle Notebook URLs (Parts 5 and 8) here for easy grading access.

## Submission Requirements

---

Submit exactly ONE Google Colab notebook. At the very top, include a student-info block with: Name, Student ID, Section, GitHub Profile, Kaggle Profile, and both datasets used.

### **Your Notebook Must Contain These Sections**

- 1. Problem Definition & Dataset Exploration   2. Task A: Cleaning & Vectorization   3. Task A: Model & Evaluation
- 4. Task A: Word Interpretation   5. Task A: Kaggle Submission   6. Task B: Cleaning & Vectorization
- 7. Task B: Model & Evaluation   8. Task B: Kaggle Submission   9. Cross-Task Comparison & vs. Logistic Regression   10. Final Reflection & Conclusion

Every table, plot, and metric must be accompanied by a short written interpretation — code with no discussion will not be accepted. Both Kaggle Notebook URLs must be working public links.

## Grading Rubric (20 Marks)

| Criteria                                                    | Description                                                                                                         |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Task A: Cleaning, Vectorization & Model                     | 4 — Correct fit-on-train-only vectorization; NB trained and evaluated with all four metrics + confusion matrix      |
| Task A: Precision/Recall Reasoning & Word Interpretation    | 3 — Sound false-positive-vs-false-negative argument; correctly extracted, sensible top words                        |
| Task A & B: Kaggle Notebook Submissions                     | 3 — Both notebooks published, public, and functionally match the Colab work (1.5 marks each)                        |
| Task B: Independent Pipeline Application                    | 4 — Correctly transferred cleaning/vectorization/model/evaluation with minimal guidance; cleaning choices justified |
| Task B: Evaluation & Independence-Assumption Discussion     | 3 — Correct metrics; genuine, specific reasoning about why performance differed from Task A                         |
| Cross-Task Comparison, vs. Logistic Regression & Conclusion | 3 — Complete comparison tables; thoughtful, evidence-based final conclusion                                         |

### Penalties

- Fitting the vectorizer on the full dataset before splitting (data leakage): marks deducted in Part 2 or 6.
- Evaluating with accuracy only (no precision/recall/F1/confusion matrix) for either task: marks deducted in Part 3 or 7.
- No genuine precision-vs-recall argument for spam (just restating definitions): marks deducted in Part 3.
- Missing, private, or non-functional Kaggle notebook links: marks deducted in Part 5 and/or 8.
- Task B copied verbatim from Task A with no adaptation or justification of cleaning choices: marks deducted in Part 6.
- Introducing algorithms not covered in this lab (e.g. SVM) in place of the requested comparison: marks deducted in Part 9.